

A Repository-Based System for Research and Publication

Architecture, evidence and iteration in a Lean research repository

WILL COOK*

30 September 2026

ABSTRACT

We describe Plectis, a repository-based system for carrying mathematical research from a question to a reviewed publication. Its public prototype organises eight mathematical programmes around shared source files and structured records. The design keeps paper statements, registered formal support and explanatory prose in separate records, allowing revisions to be checked against their cited evidence. Python tools retrieve records, store optional Lean probes, check source bindings and regenerate indexes and summaries. Maintainers decide which changes to adopt, and a journal retains entered contributions and review decisions. A worked example shows why links to correct formal proofs can survive a paraphrase that drops a stated hypothesis. A historical author-run test rejected nine of ten false edits and accepted one false completion claim. Reader benefit, discovery rate and adoption by independent laboratories remain unmeasured.

1 Introduction

Research continues across sessions, contributors and drafts. The next researcher needs to recover which question is open, what has been tried, and what supports the claims already published. A formal proof may remain correct while an edited explanation drops a hypothesis, or an unresolved step is later described as complete. Keeping the proof is therefore only part of keeping a usable research record.

Plectis is the public repository prototype studied here. It stores questions, prior sources, computations, proofs and unresolved steps alongside proposed contributions and their review. The implemented workflow is local to this repository; broader autonomous discovery and services for other scientific fields are proposals (Section 9).

The architecture separates authored research and publication records from the tools that inspect them and the indexes and summaries generated from them. Researchers and maintainers decide what to investigate and what a proposed change justifies. Figure 1 follows the resulting path from research artifacts to publication and back through contributions.

Erdős Problems supplies the questions and their public discussion, while Lean and mathlib supply the formal language and library [1, 11, 76]. Formalisation blueprints link a written argument to named Lean declarations [35, 74], and the Equational Theories Project maintains shared problem records [75]. Prove2Me combines human-audited missions, source-linked milestones and separately submitted proofs, with intermediate results available for other agents to reuse [31, Sections 3–4].

Publication review follows a paper assertion at a specified revision. Its coverage record identifies the printed statement, registered Lean supports and their stated relationship. A separate

**Authorship and AI use.* Will Cook built and directed the research infrastructure and reviewed the claims when he could. AI agents did most of the research and drafting. Cook did not independently verify every claim.

passage record binds the explanation to sources and rationale; contribution records retain entered review decisions. Together these links make the basis of a proposed revision inspectable. Agreement between the records does not establish that the prose is true.

We first describe the stored records, inspection tools and contributor responsibilities. A proposed change to a theorem's explanation then illustrates the publication checks. The revision and contribution workflows show how accepted work returns to the repository. Recorded observations and limits distinguish these implemented procedures from their unmeasured effects; Appendix B retains the mathematical calculation.

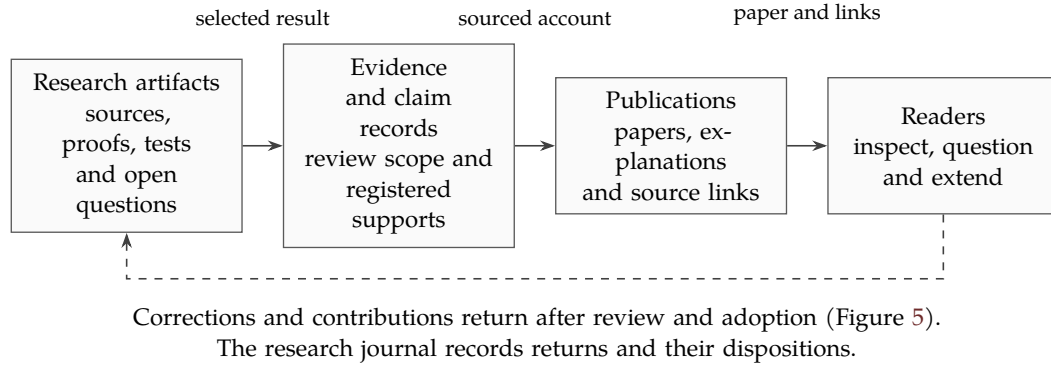


Figure 1: From accumulated work to a paper and subsequent revision. Labels name what passes between the stages; the arrows imply neither logical deduction nor automatic approval. The source and evidence records support review; they do not establish the intended meaning of the prose.

2 A repository organised around problems

2.1 Stored records and derived views

Lean and manuscript files contain the arguments. Claim records state selected assertions and what remains open; evidence records identify their registered supports; the journal retains entered returns and decisions. Builders read these authored files to produce indexes, source coordinates and status summaries. The [architecture guide](#) identifies the source of each derived view. A changed summary can therefore be investigated at its input record rather than treated as a new research result.

For publication, the coverage ledger identifies a *paper occurrence*: a labelled assertion in a particular manuscript, together with its source location and a digest of its statement. The row lists its registered Lean supports and their evidence status. Where the formal result is stronger than the printed statement, an authored relation note explains the correspondence and is bound to both statements. The passage ledger used for this systems paper has a different purpose: it records explanatory text, cited source spans, an evidence class and a rationale. Neither ledger derives the prose from a Lean proof.

2.2 Finding the relevant work

Within each problem collection, a reader may want a proof, a counterexample to a proposed extension, or the argument behind a failed approach. Exact experiments and corrected references remain beside the original question as contributions in their own right. The reader can locate these records without first compiling the whole repository. Four views answer different parts of that search (Figure 2).

A declaration is a named Lean definition, theorem or lemma. The declaration index locates its source; the dependency index records which earlier declarations the selected results use. A separate graph records authored interpretations of those declarations and their connections, while a claim registry records selected mathematical assertions, their status and the obligations still open.

For each problem: the question and literature, Lean statements and proofs, computations and certificates, claims and open obligations, failed approaches, and the paper with its supporting record.

View available without a Lean build	What it records
Declaration index	Source locations, exhaustive within its inventory.
Dependency index	Exact dependency edges for the selected starting declarations.
Interpretation graph	Selective, largely contextual accounts of declarations and their connections; not an individual reading of every proof.
Claim registry	Selected assertions, their status and remaining obligations.

Across the corpus: 8 problems, 1,903 Lean modules and 24 registered papers. The claim registry contains 160 claims in eight statuses, with 19 open obligations.

Figure 2: Contents of a problem collection and the four ways to inspect the recorded work. The counts refer to the whole corpus. Source locations, dependency edges, interpretations and claim statuses answer different questions.

Starting from a recorded assertion, a reader can therefore locate its declarations, follow the recorded dependencies and request the specified lines of source needed for a particular question. The declaration index is exhaustive within its inventory, and the dependency edges are exact for the selected starting declarations. The recorded interpretations are selective and largely contextual; they do not document an individual reading of every proof. A source link by itself records no successful execution. When a question requires a Lean check, the reader can request a focused build and inspect its recorded result.

2.3 Operations on the records

The tools provide the following operations:

Component	Input, output and responsibility
Task routing	agent_entry.py maps a stated task to instructions, relevant files and commands; the researcher chooses the question and approach.
Evidence retrieval	query_corpus.py returns problem, claim, declaration and paper-source records from the indexes.
Proof sessions	proof_workbench.py records notes and submitted Lean probes, derives verdicts from the Lean process, and binds a session claim to an accepted probe.
Statement evidence	paper_evidence.py locates each registered support at the recorded revision and checks its source and verification bindings; paper_claim_evidence.py projects their evidence status and reported gaps.
Editorial exchange	short_paper_writer.py audits manuscripts and packages proposed revisions without applying or approving the return.
Research history	research_record.py verifies the journal of entered returns and dispositions and reports the outstanding milestones of recorded outputs.
Derived views	refresh_projections.py runs builders in dependency order; check_release.py checks release contracts and freshness.

A separate self-audit, `systems_paper_evidence.py`, checks this systems manuscript's text and cited source bindings. These commands provide the operations; the researcher or agent chooses which to invoke. Their presence is not evidence of a successful run or of research quality.

Control remains with the researcher or agent; persistent state remains in files. An optional proof session stores notes, exact Lean inputs, process-derived probe receipts and a closing outcome. Its claim command requires an accepted probe, then appends a session record; it does not update the reviewed public claim registry. The research journal separately records entered returns and review dispositions, checking their sequence and hash chain. These records allow later work to inspect earlier inputs and decisions without relying on the original conversation.

2.4 Researcher, agent and maintainer responsibilities

A researcher chooses the task and the evidence needed to answer it. A coding agent can inspect files, invoke tools and prepare a proposed change using the public instructions; ordinary written arguments and explanations can also be contributed without a clone or an agent. A maintainer reviews the claim, attribution and remaining uncertainty before adoption. The contributor and reviewer may be the same agent, so this division of responsibilities does not establish independent review. Public setup and review instructions are in the [agent quickstart](#) and [contribution guide](#).

A typical iteration starts by retrieving the work relevant to a question. The researcher investigates and submits an argument, experiment, correction or manuscript change with its evidence and limitations. Review may adopt it, request revision, defer it or reject it. Accepted work updates the affected authored records; builders regenerate their dependent views, and release checks test those views for consistency. The journal preserves dispositions for structured returns explicitly entered into it. Ordinary issues, pull requests and ideas can enter review without a journal session. Sections 5 and 6 describe these two contribution routes.

3 Worked case: reviewing an explanation

The worked case concerns a theorem about infinite sums of reciprocals over a restricted set of positive integer exponents. A base-two weighted convergence condition restricts the allowable sets; under it, every infinite subset gives an irrational sum at every integer base at least two. Removing the condition would assert a solution of a more general problem that remains open. Appendix B gives the exact statement and calculation.

Suppose a contributor rewrites this theorem's explanation. Its coverage row, `res:weighted-support` in `paper_lean_coverage.json`, identifies the statement in the mathematical paper and two registered Lean supports. An explanatory passage in this systems paper is instead recorded in the `passage ledger`, with sources and a rationale. For the appendix's hereditary conclusion, these sources are the ordinary proof and a formal declaration. If the contributor drops the weighted hypothesis from that explanation, both Lean proofs and both source links can remain unchanged. The reviewer must compare the new wording with the cited statements. Section 4 explains which part of this comparison the tools check.

4 Proof and publication checks

The example has two registered formal supports (Figure 6). One gives fixed-base irrationality and the all-base conclusion for every infinite subset; the other makes the fixed-base hereditary conclusion explicit. Their roles overlap. Here the *registered support set* means the declarations listed for this paper occurrence, not every declaration on which their proofs depend. The publication check accounts for that full list even when one declaration suffices for the clause under

discussion. This is a record-consistency requirement, not an additional hypothesis of the theorem. The resolver also follows any registered named inputs and checks the identified source revisions. A result dependent on an additional named input retains that dependency; the weighted theorem above illustrates a different issue, a restriction within the proved statement. A changed statement or source must be checked against its records again. Evidence attached to one revision cannot silently support another.

These checks make omissions and changes inspectable. Figure 3 separates them from proof checking and from the judgements needed to publish a faithful account.

Relation being assessed	Check or judgement required
Formal statement and proof	Lean kernel checking.
Challenge and formal statement	Comparator checking against the selected challenge.
Paper occurrence and registered support set	Coverage audit: all registered declarations and their recorded revisions.
Sentence and declared source bytes	Sentence audit: text and source digests, with a recorded rationale.
Sentence and mathematical meaning	Author and reviewer judgement about what the sources establish.
Result and motivating question	Author and reviewer judgement about the scope of the result.
Novelty, significance and acceptance	Assessment by reviewers and the mathematical community.

Figure 3: Four checks with different objects, followed by three questions of judgement. The upper rows identify checking mechanisms, not new executions reported here. The lower rows identify review responsibilities, not evidence of independent review. A recorded rationale still requires assessment of its meaning.

Lean verifies that a proof establishes the formal statement written in the source; it does not verify whether that statement captures the intended mathematics or whether the paper describes it well. Comparator adds a separately stated challenge and a check of its selected Lean implementation under the permitted axioms [19, 20].

Each marked passage in this systems paper has a recorded text, specified source lines, an evidence class and a rationale for citing those sources. The record groups prose for checking; it need not correspond to one grammatical sentence.

For the appendix's hereditary conclusion, the sources explain why restriction to an infinite subset preserves the weighted condition. The passage checker compares cryptographic hashes of the marked text and declared sources and requires a nonempty rationale. It also rejects missing records or unbound passages. It does not assess whether that rationale justifies the text.

Changing the passage alone makes its recorded digest stale. Updating its record as well can restore byte agreement while leaving the new assertion unsupported. For example, a paraphrase could drop the support hypothesis while retaining both correct Lean links in Figure 6. A reviewer must compare its hypothesis and quantifiers with the source statement. This is an illustrative editing failure, distinct from historical edit 8, which replaced a limitation by a false completion claim (Table 1).

An entry in the declaration index with a matching type and a reachable source link identifies the recorded source; it does not establish that any particular continuous-integration run compiled the module. In the recorded design, the checking interface obtains each verdict from the specified Lean process, so callers cannot supply verdicts through that interface. This interface rule does not prevent an actor with filesystem access from changing the implementation or stored record. The protocol distinguishes contributor and reviewer roles but does not techni-

cally force a second independent mathematician. No independent human mathematical review of the corpus is recorded.

5 Publication and revision

Publication turns a result into an account a reader can use. The short paper states its conditions, explains the decisive step and points to the full argument, so the reader can recover both why the conclusion holds and where it applies. A dependency graph locates the ingredients. Choosing their order and explaining their role require reading the argument; the manuscript audit supplies no such judgement. The companion paper retains the complete proof and subsidiary cases. A correspondence link names the passage supporting each linked short-paper claim, taking the reader from explanation to calculation. The assembler inlines the companion's authored sections into a flat manuscript, retaining their cross-references. It supplies no missing mathematical step. Readers may need a hint, a worked example or the full argument. The reading guide invites them to reconstruct a decisive step or change a hypothesis, and the explanation instructions require an agent to respect the requested depth. These are authoring rules; they do not establish that a reader has understood the proof.

5.1 Work assignments and review

An integrating agent can alter the live checkout; an advisory agent returns proposed changes against supplied sources. The local names Type A and Type B denote those two permissions.

Repository authority and the assigned task are separate choices. The task may be mathematical research, proof or claim audit, literature and attribution review, or exposition.

Research returns candidate arguments, an audit returns source-located findings, a literature review returns attribution proposals, and exposition returns a clearer manuscript; each product has its own review and acceptance criteria.

An exposition assignment preserves the established hypotheses, conclusions and evidence status; a suspected mathematical defect is returned for a separate audit or research assignment.

The reviewer compares hypotheses and implication directions with the whole statement and all its registered declarations. An ordinary argument retains its actual evidence class where formalisation is incomplete.

After acceptance, the integrating reviewer applies changes to the owning manuscript sources and updates the affected evidence bindings. For an assembled companion, edits belong in its authored sections before rebuilding the flat paper. Integration also requires regenerating evidence and navigation, then inspecting the compiled pages, figures and proof links.

A writing rule learned from the revision travels with its example and limits as a candidate. Its adoption requires a reviewed change to the owning writing contract.

5.2 Proposing and reviewing a revision

A revision begins by collecting candidate results, antecedents, difficult proof transitions and unresolved questions with their source locations. The author decides which argument deserves the short paper; the checker audits source consistency and the specified manuscript rules. The exchange freezes selected manuscripts, required inputs, the coverage ledger and audit rules under a manifest of their exact bytes. Writer and integrating reviewer can therefore identify the same starting sources.

An advisory model can return a proposal against those sources; a tool-enabled agent can prepare it in a checkout. In either case the integrating reviewer decides whether the mathematics and its description remain faithful. The public writer's packet declares which manuscript paths may change and supplies its evidence records and checking tools as read-only inputs.

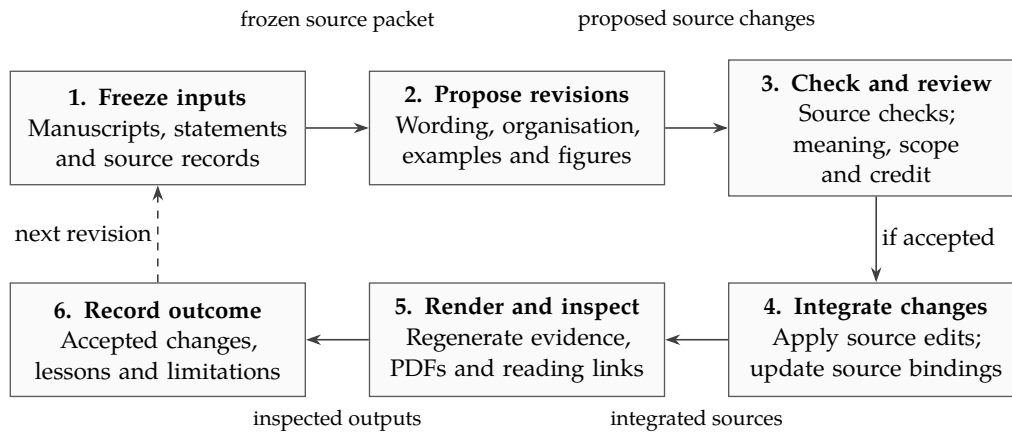


Figure 4: The exposition workflow, read in numerical order, with the exchanged objects labelled. A proposal may be revised or declined at step 3; only accepted changes proceed to integration. The steps name responsibilities, which may be performed by the same agent. Rendering and inspection remain separate from source checks.

Before that decision, the return checker tests packet identity, permitted paths, declared statement changes and retained labels, citation keys and bibliography entries. It audits the proposed manuscript in a temporary copy of the frozen sources, leaving the live repository unchanged. It neither applies the proposal nor executes returned programs. Declaring a statement change makes the review obligation explicit; it does not discharge it.

The packet and audit stages in Figure 4 are implemented by `short_paper_writer.py` and `paper_refinement.py`.

A return has two possible lasting effects: accepted text changes the paper; an accepted general lesson changes the guidance supplied to later writers. The latter needs its own review, so a local improvement can be retained without becoming a new rule. This process revises documents and instructions, not model weights. The procedure supplies no measurement of reader benefit or autonomous discovery.

6 An open route for contributions

A contributor may send a mathematical idea or reference without a clone, or return a patch from a recorded public commit. For executable work the return identifies the changed files, commands, results, resource use and surviving limitations, with the people and tools responsible. Figure 5 separates the proposed change from replay, review and adoption. The return protocol calls for reproduction at the submitted base before reconciliation with current main, with separate credit for a substantive integration repair.

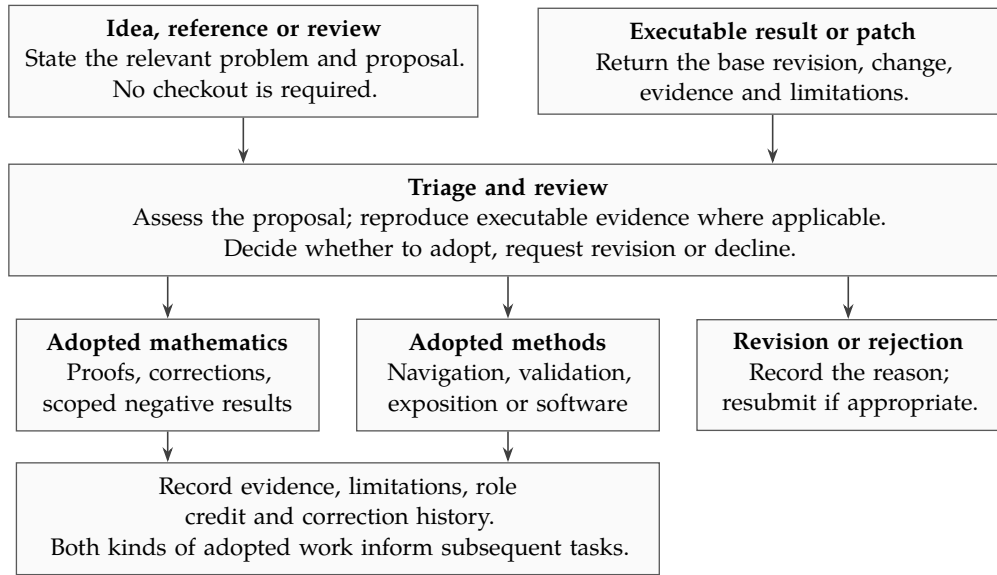


Figure 5: Two entry routes and three possible review outcomes. Repository adoption applies to mathematical and methodological contributions; it establishes neither independent review nor acceptance by the wider mathematical community.

The research journal records explicitly entered returns and dispositions in an append-only, hash-linked history, while a correction may withdraw something previously endorsed. Ordinary issues and pull requests need not become journal events. Discovery investigates a question, while stewardship compares the return with prior work, groups related results, reconciles affected papers and selects the next question. A changed result or appraisal can prompt another task. The public workbench leaves that choice to the researcher or agent; the journal records entered decisions and does not schedule further research. The journal preserves the entered return and its recorded disposition. Judging what was learned requires comparison with earlier work and inspection of the returned evidence.

Generalising a local result requires a separate literature and library comparison, a proof of the proposed extension, and a second useful instance before an expert takes it to an upstream project [100]. That route is not automated and has produced no reported Mathlib contribution in the supplied record. We distinguish credit for an idea from credit for its implementation, formalisation or exposition, using CRediT as a role vocabulary rather than an authorship decision [104]. Later use adds an attributed dependency or correction, with neither a scalar impact score nor a financial contract inferred from that entry. A new problem collection likewise needs a sourced question, explicit status, inspection routes and a review owner, which must still be assembled in several steps.

7 Recorded observations

The frozen inventory contains 649 paper occurrences, of which 604 have exact or specialising Lean support; 20 depend on a named input and 25 have no recorded Lean support. Of the 604 exact-or-specialising occurrences, 580 are recorded as compared and 24 as queued. These counts classify statement occurrences. Exact and specialising support match a paper statement directly or establish it by specialization. Named-input entries retain an additional assumed result; an entry without Lean support may still have an ordinary mathematical proof. Neither group is counted among the fully covered entries.

The historical publication-evidence record reports that nine of the ten deliberately false edits were rejected and one escaped. The edits were authored by the checker’s author, the original run logs were not retained, and the other nine edits were not rerun against the extended checklist. The edits were applied separately, with the baseline restored between trials. The escaped edit changed a description of an unresolved step from “does not supply” to “completes”, and the publication checks accepted it.

Edit	Change	First detector	Outcome
edit 1	Conditional result labelled proved	Projection freshness	rejected
edit 2	Open-boundary clause deleted	Boundary wording	rejected
edit 3	Declaration coordinate moved	Source coordinates	rejected
edit 4	Paper source link retargeted	Source coordinates	rejected
edit 5	Claim assigned to two families	Registry structure	rejected
edit 6	Module count hand-edited	Projection freshness	rejected
edit 7	native_decide introduced	Proof-trust policy	rejected
edit 8	“does not supply” changed to “completes”	None	escaped
edit 9	Open-problem title mangled	Paper anchoring	rejected
edit 10	Orientation byte budget exceeded	Byte budget	rejected

Historical trial: edit 8

The false completion clause passed all recorded checks.

Later witness: baseline and edit 8

Baseline accepted, edit 8 rejected; the other nine edits were not rerun.

Table 1: The ten historical edits and their recorded first detectors; “proof-trust policy” refers to this repository’s permitted proof mechanisms, not to a general unsoundness claim about native_decide.

After the escape, the follow-up checked the intact baseline and that particular edit against an added check, leaving the other edits untested under the repair and supplying no post-repair ten-of-ten result. In that recorded follow-up, the post-repair witness accepts the current README and rejects a test copy containing the false clause. The sample comes from one corpus and one author’s selection of errors, with neither a manual-review control nor an ordinary continuous-integration comparator. The study locates a coverage boundary. Its nine rejections do not estimate how reliable the checker is.

On 23 September 2026, a maintainer-operated agent accepted one architecture handoff into a local integration history after focused checks of return routing and architecture packaging. The contributor and reviewer were the same agent. The receipt records no public pull request, human review or independent outside clone replay.

For readers, the proposed comparison holds the mathematical task fixed and contrasts an ordinary repository, an information-equivalent static briefing and the navigation tools. It would record understanding, severe errors and the costs of preparation, use, review and repair, including missing returns, timeouts and disagreements. No independent writing comparison, blind grading run or cold-reader experiment is reported here, and no comparative reader result is reported. General reliability, reader understanding and mathematical transfer remain unmeasured.

8 Related work

The closest comparisons concern the objects a system records and the work it lets contributors perform. Lean and mathlib provide formal statements, proofs and a maintained library; blueprints connect declarations to exposition [11, 76, 35]. Prove2Me provides a more direct comparison for agent-assisted collaboration. Appendix C retains the wider source map.

Prove2Me. We use its published version 2, dated 31 August 2026; the first version appeared on 28 August 2026 [31]. Its basic unit is an immutable theorem statement, stored separately from the proofs submitted for it. A proof is checked against the target type in a pinned verification environment. A *proof-sketch* derives a target using imported theorem statements, some of which may remain open. Contributors can prove those children separately; the target becomes verified when the required children are verified. Immutability makes those local proofs composable without changing their targets [31, Sections 3.1–3.2 and 4.2].

Human review is part of Prove2Me's design. A mission fixes a human-audited core of goals, definitions and milestones; each milestone pairs a source statement with its attested canonical formalisation. For read-back, an independent agent receives the Lean declaration and dependent definitions without the original source; a human compares its mathematical rendering with that source. Natural-language accounts accompany statements and proofs, statement descriptions support library search, and imports give contributors reuse credit. The discussion record also includes a disproved statement followed by a corrected replacement [31, Sections 3.3 and 4.3–4.5]. Explanation, reuse, correction and shared agent work are therefore established parts of this comparison.

For correspondence with a source, the closest analogue is Prove2Me's milestone: both designs identify the mathematical statement whose formalisation is being reviewed. Our coverage record follows a particular paper occurrence and can list several overlapping formal supports, with a relation note explaining their connection to the printed claim. It is not a proof-sketch: listing those supports does not compose proof terms or discharge open lemmas. A separate passage record binds the explanation to sources and rationale, while entered contribution records retain revision decisions and role credit. The focus here is the consistency and review of these publication records across revisions. This difference of focus establishes neither priority for persistent research records nor a measured advantage over Prove2Me.

Polymath, BOINC and GIMPS supply precedents for small mathematical contributions and donated compute [99, 97, 98], while the Carleson blueprint shows formalisation organised into publicly claimable tasks [87].

Our fixed-order regeneration and freshness checks address the build-system problem of keeping derived views consistent with their inputs [48]. The implementation runs an explicit ordered list of builders, then checks the resulting views; it is not a general incremental scheduler. Source bindings and the journal record artifact provenance, including the inputs and revision to which a claim or entered return refers. These records serve inspection and reproducibility, with research efficiency and semantic correctness left to separate evaluation.

9 Limits

This paper reports no comparative formalisation result or autonomous-optimisation experiment. Transfer to unseen mathematics and understanding by independent human readers are unresolved. The record supplies no controlled evidence that the contribution machinery improves the conversion from compute to mathematics or increases discovery rate. It also supplies no evidence that a new project can adopt the architecture cheaply or that an agent using it outperforms one using a good README. Full environment scans and initial builds still have costs, and an open corpus cannot establish that a model has never encountered its material.

The project remains maintainer-centred, with scarce review capacity and contestable credit; openness alone does not equalise resources or resolve conflicts and appeals. Broad mathematical acceptance is exogenous to this repository and cannot be granted by its maintainer. Returned code is untrusted, and expensive or privileged continuous-integration jobs must not execute fork code with repository secrets [105]. A volunteer-compute service and extensions to physical science remain proposals requiring their own validation, resource and safety governance.

The Advisory Group on Mathematics and Artificial Intelligence at IAS recommends distinguishing papers fully understood by a responsible mathematician from AI output that its human prompters do not understand, and asks AI laboratories to support subsequent community-led understanding [10]. Its 29 September 2026 statement gives concrete release expectations for attribution, exposition, formalisation and disclosure of the research process.

Several of these proposed release practices have counterparts in the records described here [10, Section 2.B, Step I]: the short paper and companion paper explain and attribute the argument, the paper-to-Lean ledger records formal correspondence and its limits, and the challenge files and generated formalization.yaml expose the comparison inputs. Those records make the result inspectable; they do not certify a mathematician’s understanding. The architecture alone does not establish complete per-result model, prompt, time and cost disclosure, an account of campaign-wide selection and unsuccessful attempts, or an independent scholarly deposit. The further recommendations on funding community-led understanding and equitable model access address AI laboratories; this prototype does not demonstrate those institutional outcomes.

The present contribution is an inspectable route through the records: a reader can recover the hypothesis, argument and support of a stated result, and identify where a source check ends and mathematical review begins.

A Inspection and reproduction

The initial unified manuscript was prepared from the frozen packet at ced96e58b337, a local integration commit rather than public main. A 29 September state report lists this manuscript as *Problem-Sized Lean Worlds*. Repository links in this revision use commit 992cf60a27ef; the Comparator links below identify their separate recorded runs. No new Lean or Comparator run was performed for this revision. This paper supersedes *From a Cold Clone to a Proof Receipt* and *From Spare Compute to Cumulative Mathematics*. Their manuscripts remain available as historical accounts of navigation, proof-check records, contribution protocol and governance.

A.1 Earlier design accounts and current status

[From a Cold Clone to a Proof Receipt](#) (August 2026) explains the declaration, dependency, interpretation and claim views and the optional recorded proof session. The current task entry and the inspected implementation are described in Section 2; the earlier paper’s inventory and audit figures belong to its dated snapshot.

[From Spare Compute to Cumulative Mathematics](#) (September 2026) explains contribution roles, original-base reproduction, later integration and role credit. Section 6 describes the current public return procedure and distinguishes ordinary submissions from explicitly entered journal events. Volunteer-compute services, inexpensive adoption by new laboratories and improvements in discovery rate remain proposals or unmeasured outcomes. These two manuscripts are historical design sources, not additional current system specifications.

The weighted row in `docs/paper_lean_coverage.json` names `divisibilityWeightedClaim` and `finitePrimeWeighted_fixedBase_hereditary`, in namespace `ErdosProblems.Erdos257.PaperCompleteR8`. Appendix B follows the ordinary proof in [2]; these entries record formal support for the conclusion, not a line-by-line comparison of that argument with the Lean proofs.

Its Comparator records are [run 35544127144](#) and [run 35624228171](#); the latter also records PalomarCorpus/E257av for the reciprocal antecedent. Both runs are recorded as compared; Palomar is prepared and not submitted.

For Erdős’s reciprocal-summability criterion, the formal declaration is [Erdos249257.irrational_erdosSupportSeries_of_summable_reciprocal](#), with Comparator entry PalomarCorpus/E257av recorded as compared.

The four declarations supporting the reported 1041 refutation have namespace Erdos1041. Counterexample: [erdos1041_counterexample](#), and the [HausdorffLength.lean](#) declarations [erdos1041_counterexample_hausdorff](#), [erdos1041_hausdorff_negation](#) and [erdos1041_hausdorff_answer_false](#). The coverage row gives exact-or-stronger Lean support, with Comparator pending.

In a complete checkout, `python3 scripts/systems_paper_evidence.py` checks this paper’s source bindings, and `python3 scripts/build_systems_paper_counts.py` checks the generated inventory. The record at [docs/publication_evidence.json](#) holds the historical matrix and the limited follow-up, while [docs/publication_contract.json](#) declares the publication checks. Formal entry points include [Erdos249257.lean](#) and [.github/workflows/lean.yml](#); the curated assertions are in [docs/claims.json](#), and [scripts/check_release.py](#) runs the release gate. The public writer’s native packet freezes manuscript inputs, selected records and the tools used to inspect them; its input list does not include a Lean source tree. Source-binding validation does not replay Lean, Comparator or the full repository release checks.

The protected snapshot partitions 649 occurrences into 475 exact, 129 specialising, 20 named-input and 25 unsupported-by-Lean entries. The journal summary at head d3ad85269386 records 7 rounds (2 sealed), 25 returns (18 admitted and 7 unreviewed); earlier private-history rounds are not a complete public journal. The interpretation inventory covers 139,818 of 152,386 authored theorem-like declarations (91.8%). These are selective, largely contextual interpretations, not individually reviewed proofs. Public reading starts at the [reading guide](#). [Agent instructions](#) route source inspection, explanation and reproduction; the [workbench guide](#) describes optional recorded sessions. [Contributing](#) and the [credit policy](#) govern returns. Cite repository software using [CITATION.cff](#) at the chosen commit or release tag; cite mathematical results by the paper and version actually used.

B Calculation for the worked theorem

For an infinite set A of positive integers and an integer $b \geq 2$, write

$$X_A(b) = \sum_{a \in A} \frac{1}{b^a - 1}.$$

The question is when a condition on the exponents forces this number, and the corresponding sum over every infinite subset of A , to be irrational.

The condition below rewards exponents divisible by large powers of a fixed collection of primes. For the prime 2 alone, the relevant part of $a = 2^k m$, with m odd, is 2^k . For a finite nonempty set P of primes, this becomes $h_P(a) = \prod_{p \in P} p^{v_p(a)}$, where $v_p(a)$ is the exponent of p in the prime factorisation of a .

The weighted-support theorem of the Problem 257 paper states that, if

$$W_{2,P}(A) := \sum_{a \in A} \frac{h_P(a)}{a(2^{h_P(a)} - 1)} < \infty, \quad (1)$$

then $X_B(b)$ is irrational for every integer $b \geq 2$ and every infinite $B \subseteq A$ [2, Theorem 1.2].

For a fixed base b , write $W_{b,P}(A)$ for the sum in (1) with 2 replaced by b . The fixed-base version assumes $W_{b,P}(A) < \infty$ and gives the same hereditary conclusion at that base. The calculation below gives the ordinary argument; Figure 6 identifies its two registered Lean supports.

This is a sufficient condition on the support; the question for an arbitrary infinite set of exponents in base two remains open.

Erdős had already stated that $\sum_{a \in A} 1/a < \infty$ suffices for this all-base irrationality conclusion [3, p. 222].

The source paper credits Duverney and Tachiya for an arithmetic-progression averaging argument of the kind used to select a small positive displacement [4, Section 2, (2.3)–(2.9)].

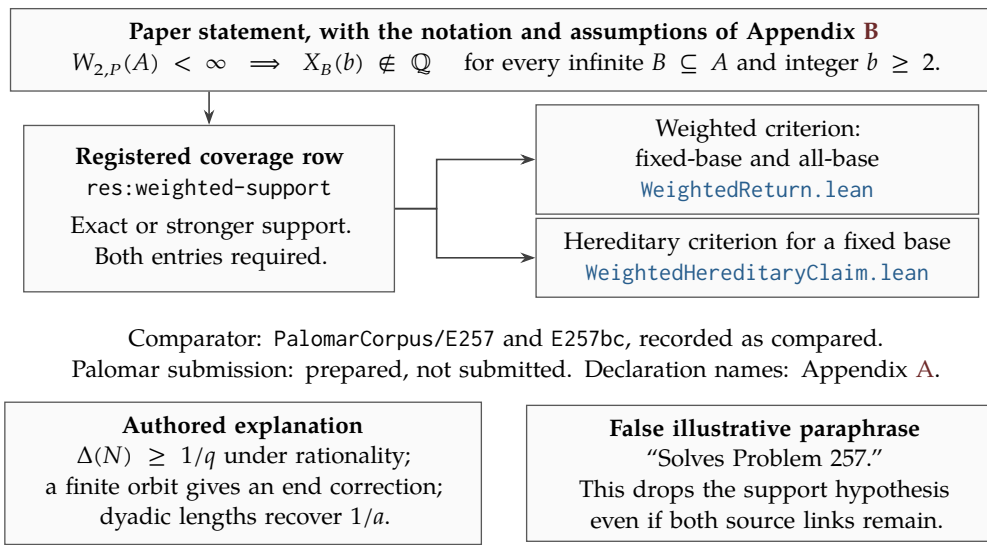


Figure 6: The arrows associate a paper statement with its coverage row and the two registered supports; they do not represent deductions between these records. The support hypothesis still needs mathematical explanation. The false paraphrase is illustrative, distinct from historical edit 8 in Table 1.

This appendix retains the ordinary calculation from the Problem 257 paper used by the worked case in Section 3.

The distinction from the reciprocal-summability criterion is visible in the source paper’s example

$$A_* = \{2^k m : k \geq 1, m \text{ odd}, 1 \leq m \leq 2^k\}, \quad P = \{2\}.$$

Within the layer $v_2(a) = k$, reciprocal summation discounts each odd m by 2^{-k} ; weighted summation discounts it by $(2^{2^k} - 1)^{-1}$.

To compare the layers, put $S_r = \sum_{1 \leq m \leq 2^r, m \text{ odd}} 1/m$. Grouping the odd integers into intervals $[2^j, 2^{j+1})$ gives $r/4 \leq S_r \leq r$ for integers $r \geq 2$.

The reciprocal contribution of each disjoint layer is at least $2^{-k} S_{2^k} \geq 1/4$. Its weighted contribution satisfies

$$\frac{S_{2^k}}{2^{2^k} - 1} \leq \frac{2^k}{2^{2^k} - 1} \leq 2^{1-k},$$

so the layer contributions have a convergent geometric majorant in the weighted sum.

Thus A_* lies beyond the reciprocal-summability criterion but satisfies (1). This is the source paper's ordinary calculation, with no separate entry in the paper-to-Lean ledger [2, calculation following the weighted proof].

A positive displacement. Suppose that $X_A(b) = p/q$ with integers p, q and $q > 0$. For a positive integer N , multiplication by $b^N - 1$ removes the complete geometric blocks from each summand, leaving

$$\Delta_{b,A}(N) = \sum_{a \in A} \frac{b^{N \bmod a} - 1}{b^a - 1} = (b^N - 1)X_A(b) - J_{b,A}(N), \quad N \geq 1, \quad (2)$$

where $J_{b,A}(N) = \sum_{a \in A, a \leq N} \sum_{j=1}^{\lfloor N/a \rfloor} b^{N-ja}$ is an integer.

Every summand is nonnegative, and one is positive because the infinite set A contains an exponent larger than N . Hence $q\Delta_{b,A}(N)$ is a positive integer, and $\Delta_{b,A}(N) \geq 1/q$.

We obtain a contradiction by finding a multiple $N = tQ$ with smaller displacement. Divisibility helps because the summand indexed by a vanishes whenever $a \mid Q$.

Choose a finite nonempty $F \subset A$ outside which the weighted sum is less than ε , and make Q a multiple of every element of F . Those terms then disappear for every t ; we must control the remaining terms on average.

A finite orbit and its unfinished end. With $b = 2$, $Q = 4$ and $a = 6$, the residues $tQ \bmod a$ are 4, 2, 0, and their mean displacement is

$$\frac{1}{3} \left(\frac{15}{63} + \frac{3}{63} + 0 \right) = \frac{2}{21} \leq \frac{1}{9}.$$

For a general exponent a , put $g = \gcd(a, Q)$. The residues run through the multiples of g in a cycle of length a/g . Replacing each numerator $b^r - 1$ by b^r bounds the sum over one cycle by $1/(b^g - 1)$. There are at most $T/(a/g)$ complete cycles and one unfinished cycle among the first T terms. Dividing their total by T gives

$$\frac{1}{T} \sum_{t=1}^T \frac{b^{tQ \bmod a} - 1}{b^a - 1} \leq \underbrace{\frac{g}{a(b^g - 1)}}_{\text{complete cycles}} + \underbrace{\frac{1}{T(b^g - 1)}}_{\text{unfinished cycle}}. \quad (3)$$

The first term has the shape of the weighted hypothesis once suitable prime powers have been put into Q . The second has lost the factor $1/a$. Summability of the first therefore gives no licence to sum the unfinished-cycle bounds over all exponents. This is the difficulty that a single progression average leaves unresolved.

A second average restores the missing factor. The source proof averages over the M lengths $T = 2^j$, $M \leq j < 2M$. Thus M counts the lengths being averaged, whereas T counts the terms in one progression average. For nonnegative weights α_a indexed by positive integers, with $\sum_a \alpha_a/a < \infty$, it uses

$$\sum_{j=M}^{2M-1} 2^{-j} \sum_{a \leq Q2^j} \alpha_a \leq 2Q \sum_a \frac{\alpha_a}{a}. \quad (4)$$

Indeed, a fixed a enters only when $2^j \geq a/Q$, and the sum of the admissible 2^{-j} is at most $2Q/a$. This recovers the missing factor before summing over a . Dividing by the number M of lengths then reduces the cost from $2Q$ to $2Q/M$.

Balancing the two costs. The modulus must cancel F while making the remaining exponents amenable to one of two estimates. Fix a positive common multiple L of F , put $p_* = \max P$, and, for $H \geq 2p_*$, set

$$Q = L \prod_{p \in P} p^{\lfloor \log_p H \rfloor}, \quad G = \lfloor H/p_* \rfloor.$$

If $h_p(a) \leq H$, then $h_p(a) \mid Q$. Since $n/(b^n - 1)$ decreases for positive integers n , the complete-cycle term in (3) is bounded by $h_p(a)/[a(b^{h_p(a)} - 1)]$. Its sum outside F is less than ε .

If $h_p(a) > H$, then $\gcd(a, Q) \geq G$: either every P -prime-power component of $h_p(a)$ is at most H , so $h_p(a) \mid Q$, or one exceeds H , and its truncated component in Q is greater than $H/p_* \geq G$. Summing (3) over these exponents up to QT costs at most

$$\frac{G(1 + \log(QT)) + Q}{b^G - 1} :$$

the harmonic sum supplies $1 + \log(QT)$, and at most QT unfinished-cycle terms supply Q .

Increasing M therefore has two effects. It reduces the first group's unfinished-cycle cost Q/M , but increases the second group's cost through $\log T < 2M \log 2$. We need both $Q/M \rightarrow 0$ and $GM/b^G \rightarrow 0$, not merely a long average. With F and L fixed, $Q \leq LH^{|P|}$ and $G = H/p_* + O(1)$. The choice $M = \lfloor b^{G/2} \rfloor$ meets both requirements.

The source proof combines these estimates into the finite mean

$$\begin{aligned} & \frac{1}{M} \sum_{j=M}^{2M-1} \frac{1}{2^j} \sum_{t=1}^{2^j} \Delta_{b,A}(tQ) \\ & \leq \varepsilon + \frac{2QW_{b,P}(A)}{M} + \frac{G(1 + \log Q + 2M \log 2) + Q}{b^G - 1} + 4 \cdot 2^{-M}. \end{aligned} \tag{5}$$

The terms bound, respectively, the weighted tail over complete cycles, its unfinished cycles, the exponents with large gcd, and the exponents beyond QT . The second term uses (4) with $\alpha_a = \mathbf{1}_A(a)/(b^{h_p(a)} - 1)$; the last uses the geometric tail bound $4/T$. After ε , F and L have been fixed, every term after ε tends to zero as $H \rightarrow \infty$ [2, Section 2, weighted-support proof].

The left side of (5) is an average with nonnegative weights summing to one: each length has weight $1/M$, distributed equally among its 2^j terms. For large H , at least one displacement is therefore less than 2ε . Taking $0 < \varepsilon < 1/(2q)$ contradicts the lower bound $1/q$. This argument gives no rate of decay in N . Passing to an infinite $B \subseteq A$ only decreases the weighted sum, and $b^{h_p(a)} - 1 \geq 2^{h_p(a)} - 1$ gives the all-base conclusion from the binary hypothesis [2, Section 2].

B.1 Other mathematical attribution and scope examples

The relation to the motivating problem must also survive compression: the Problem 269 record distinguishes distinct running-LCM heights from the heights counted with multiplicity in the original question [28].

A result about the former is not automatically a result about the latter.

When a statement is sharpened, merged or moved, its complete support set and any named input must move with it, and the statement relation needs renewed review.

A new arrangement of a proof does not make its ingredients new.

The totient record refines Coons's non- k -regularity antecedent and distinguishes Martin's separation theorem from its own Chinese-remainder and Dirichlet argument traced to Yazdani [13, 14, 15, 16].

The running-LCM record retains Fan's public priority for the two-prime case despite an independently reported derivation, and credits its Hecke–Mahler antecedents separately [28, 29, 26, 27].

For Problem 1041, the public degree-seven construction is ani's; the recorded Lean result refutes the exact Formal Conjectures statement, while correspondence with the 1958 curve-length wording remains unreviewed [23, 77, 24, 22, 21].

Appendix A names its four recorded Lean declarations; Comparator is pending.

The other seven targets remain unresolved here.

C Sources by role

Source family	Role in the three source accounts
Formal foundations	Lean and mathlib; library maintenance and contributor practice [11, 76, 92, 100]
Source and statement checks	Comparator, kernel replay, vacuity, linters and property tests [19, 20, 52, 43, 51]
Formal-to-prose tools	Blueprints, source documentation, translation and refactoring [35, 74, 37, 36, 61, 70]
Retrieval and interaction	Premises, proof states and exact declaration neighbourhoods [39, 88, 90, 91, 89, 93]
Proof automation	Premise selection and tactic search [49, 50, 59]
Library structure	Declaration networks and repository knowledge graphs [45, 46, 95]
Assumptions and reuse	Proof mining, generalisation, lemma mining and theory exploration [41, 42, 53, 54]
Growing proof knowledge	Library learning, evolving procedures and candidate statements [55, 56, 57, 58, 62, 83]
Research agents	Executable objectives, proof cost and evolving tasks [78, 79, 30, 25, 60]
Agent collaboration	Bounties, planner/worker state, retrieval and compiler feedback [80, 81, 82, 31, 44, 75]
Evaluation	Research budgets, proof engineering and documentation tasks [84, 71, 72, 73, 106, 102, 103]
Validity and intended meaning	Research case studies, interactive proving and public accounting [18, 32, 40, 33]
Public infrastructure	Problem sources, formal conjectures, verification and collaboration [1, 77, 101, 87, 99, 97, 98, 104, 105]
Writing and maintenance	Mathematical exposition, build dependencies and provenance [85, 69, 48, 47, 94]
Understanding and reuse	Explanatory generality, digestion and research possibilities [5, 6, 9, 86, 7, 63, 96]
Different community views	Declarations, dissent, benchmarks, research infrastructure and pace [8, 64, 65, 66, 34, 67, 68]
Mathematical antecedents	Weighted support, totient ranks and arithmetic separation [2, 3, 4, 13, 14, 15, 16]
Other mathematical records	Running LCM, lemniscates and cross-problem connections [28, 29, 26, 27, 12, 38, 22, 21, 23, 24, 17]

Figure 7: Sources grouped by the roles for which the original accounts cite them, rather than by a performance ranking; the accompanying mathematical attribution distinguishes original results, refinements and independent rediscovery.

References

- [1] T. F. Bloom, *Erdős problems*, erdosproblems.com, accessed September 2026.
- [2] W. Cook, *Weighted and Covered Supports for Mersenne Subseries*, short paper on Erdős Problem 257, September 2026, [PDF](#).
- [3] P. Erdős, *On the irrationality of certain series*, Math. Student 36 (1968), 222–226 (issued 1969), [scan](#).
- [4] D. Duverney and Y. Tachiya, *Refinement of the Chowla–Erdős method and linear independence of certain Lambert series*, Forum Math. 31 (2019), 1557–1566, [DOI](#).
- [5] B. Kra, *Deep theorems were scarce and difficult and so became an effective mechanism to identify deep thought. AI has broken this system*, guest post on *What’s new*, 13 September 2026, [blog post](#).
- [6] H. Cohn, *The technical debt of AI-generated mathematics*, guest post on *What’s new*, 15 September 2026, [blog post](#).
- [7] T. Tao, thread on mining open problems, Mathstodon, 8 September 2026, [thread](#).
- [8] Twenty-five Fields Medallists, *A severe misalignment of AI in mathematics*, declaration posted on *What’s new*, 11 September 2026, [blog post](#).
- [9] G. Sanderson, *If math is more than proof, we need to better celebrate the rest of it*, guest post on *What’s new*, 18 September 2026, [blog post](#).
- [10] Advisory Group on Mathematics and Artificial Intelligence at IAS, *Responsible Release of AI-Generated Mathematics*, 29 September 2026, [recommendations](#), [PDF](#).
- [11] L. de Moura and S. Ullrich, *The Lean 4 Theorem Prover and Programming Language*, in *Automated Deduction, CADE 28*, Lecture Notes in Computer Science 12699, 2021, pp. 625–635, [DOI](#).
- [12] P. Erdős and R. L. Graham, *Old and New Problems and Results in Combinatorial Number Theory*, Monographies de L’Enseignement Mathématique 28, 1980, p. 61.
- [13] W. Cook, *Integral Relations among Totient Sections*, short paper on Erdős Problem 249, revised 30 September 2026, [PDF](#).
- [14] M. Coons, *(Non)Automaticity of number theoretic functions*, J. Théor. Nombres Bordeaux 22 (2010), 339–352, [DOI](#), Theorem 3.2.
- [15] G. Martin, *Simultaneous inequalities among values of the Euler phi-function*, 2006, [arXiv:math/0603053v1](#), Theorem 1.
- [16] S. Yazdani, *Multiplicative functions and k-automatic sequences*, J. Théor. Nombres Bordeaux 13 (2001), 651–658, [Numdam](#).
- [17] W. Cook, *Reading Eight Erdős Problems Together*, synthesis paper, September 2026, [source](#), [including the periodic-chain theorem](#).
- [18] T. Feng, T. Trinh, G. Bingham, et al., *Semi-Autonomous Mathematics Discovery with Gemini: A Case Study on the Erdős Problems*, 2026, [arXiv](#).
- [19] Lean FRO, *Comparator*, 2025, [GitHub](#).

- [20] ammkrn, *nanoda_lib*, an independent type checker for Lean 4, [GitHub](#).
- [21] T. F. Bloom, *Erdős Problem #1041*, [erdosproblems.com](#), accessed September 2026.
- [22] P. Erdős, F. Herzog and G. Piranian, *Metric properties of polynomials*, J. Analyse Math. 6 (1958), 125–148, [DOI](#).
- [23] ani, post in the Problem 1041 discussion thread, 7 September 2026, [erdosproblems.com forum](#).
- [24] Formal Conjectures, pull request 6505, *Erdős 1041: mark solved with answer(False) and link a formal proof*, merged 23 September 2026, [GitHub](#).
- [25] T. Zheng, X. Wu, Z. Zhang, et al., *Dream-RSI: Recursive Self-Improvement through Evolving Worlds*, 2026, [arXiv:2609.14858](#).
- [26] J. H. Loxton and A. J. van der Poorten, *Arithmetic properties of certain functions in several variables III*, Bull. Austral. Math. Soc. **16** (1977), 15–47.
- [27] Y. Bugeaud and M. Laurent, *Transcendence and continued fraction expansion of values of Hecke–Mahler series*, Acta Arith. **209** (2023), 59–90.
- [28] W. Cook, *Distinct running least common multiples*, short paper on Erdős Problem 269, September 2026, [PDF](#).
- [29] S. Fan, comment on Erdős Problem #269, [erdosproblems.com forum](#), 26 June 2026, [forum post](#).
- [30] N. Patel, A. Rammal, A. Hayat, R. Munos, and J. Kempe, *Learning to Discover Interesting Mathematics*, [arXiv:2609.28603v1](#), 2026, [arXiv:2609.28603v1](#).
- [31] S. Chen, K. Marwaha, X. Lu, H. Yuen, and T. Peng, *Prove2Me: An Open Collaborative Platform for Scaling Math Formalization*, 2026, [arXiv](#).
- [32] T. Feng, T. H. Trinh, G. Bingham, et al., *Towards Autonomous Mathematics Research*, 2026, [arXiv](#).
- [33] T. Tao, *AI contributions to Erdős problems*, [GitHub](#), accessed September 2026.
- [34] T. Ringer, *Becoming a benchmark*, guest post on *What’s new*, 17 September 2026, [blog post](#).
- [35] P. Massot, *leanblueprint*, plasTeX plugin for Lean formalisation blueprints, 2020, [software repository](#).
- [36] Y. Zhang, Y. Sun, T. Suzuki, J. D. Lee, and F. Liu, *LeanMarathon: Toward Reliable AI Co-Mathematicians through Long-Horizon Lean Autoformalization*, 2026, [arXiv:2606.05400](#).
- [37] S. Hattori, T. Matsuzaki, and M. Fujiwara, *Natural Language Translation of Formal Proofs through Informalization of Proof Steps and Recursive Summarization along Proof Structure*, in *Proceedings of the 18th International Natural Language Generation Conference*, 2025, pp. 376–389, [ACL Anthology](#).
- [38] S. Akiyama and H. Kaneko, *Multiplicative analogue of Markoff–Lagrange spectrum and Pisot numbers*, Adv. Math. **380** (2021), 107547, p. 3, reporting the theorem of A. Dubickas, J. Number Theory **117** (2006), 222–239, [arXiv:1911.06170v6](#).
- [39] K. Yang et al., *LeanDojo: Theorem Proving with Retrieval-Augmented Language Models*, NeurIPS 2023, Datasets and Benchmarks Track, [arXiv:2306.15626](#).

- [40] C. Li, Z. Lai, D. An, J. Hu, and Z. Wen, *Advancing Mathematical Research via Human-AI Interactive Theorem Proving*, 2025, [arXiv:2512.09443v2](#).
- [41] U. Kohlenbach, *Applied Proof Theory: Proof Interpretations and their Use in Mathematics*, Springer Monographs in Mathematics, Springer, Berlin, 2008.
- [42] A. Gandhi, A. R. Tadipatri, and T. Gowers, *Automatically Generalizing Proofs and Statements*, in *16th International Conference on Interactive Theorem Proving*, LIPIcs 352, 2025, pp. 12:1–12:18, [DOI](#).
- [43] Lean community, *Batteries*, the unusedArguments linter in Batteries/Tactic/Lint/Misc .lean, [GitHub](#), accessed September 2026.
- [44] J. Xin, A. Schneidman, C. Cummins, K. Ram, S. Ganesh, and J. Limperg, *AXLE: A Cloud Infrastructure for Lean 4 Theorem Proving Utilities*, 2026, [arXiv:2606.26442](#).
- [45] X. Li, N. Peng, S. Severini, and P. Shafto, *The Network Structure of Mathlib*, 2026, [arXiv:2604.24797](#).
- [46] F. Huch, *Structure in Theorem Proving: Analyzing and Improving the Isabelle Archive of Formal Proofs*, extended abstract, 2022, [arXiv:2209.13305](#).
- [47] J. de Kleer, *An assumption-based TMS*, Artificial Intelligence 28 (1986), 127–162, [DOI](#).
- [48] A. Mokhov, N. Mitchell, and S. Peyton Jones, *Build Systems à la Carte*, Proc. ACM Program. Lang. 2 (ICFP), 2018, article 79, [DOI](#).
- [49] T. Zhu, J. Clune, J. Avigad, A. Q. Jiang, and S. Welleck, *Premise Selection for a Lean Hammer*, 2025, revised 2026, [arXiv:2506.07477](#).
- [50] J. Limperg and A. H. From, *Aesop: White-Box Best-First Proof Search for Lean*, in *Proceedings of the 12th ACM SIGPLAN International Conference on Certified Programs and Proofs*, 2023, pp. 253–266, [DOI](#).
- [51] Lean community, *Plausible*, a property testing framework for Lean 4, [GitHub](#), accessed September 2026.
- [52] H. Chockler, A. Gurfinkel, and O. Strichman, *Beyond Vacuity: Towards the Strongest Passing Formula*, Form. Methods Syst. Des. 43 (2013), 552–571, [DOI](#).
- [53] C. Kaliszyk and J. Urban, *Lemma Mining over HOL Light*, LPAR-19, LNCS 8312, 2013, pp. 503–517, [DOI](#).
- [54] M. Johansson, D. Rosén, N. Smallbone, and K. Claessen, *Hipster: Integrating Theory Exploration in a Proof Assistant*, CISM 2014, LNCS 8543, pp. 108–122, [DOI](#).
- [55] K. Ellis et al., *DreamCoder: Growing Generalizable, Interpretable Knowledge with Wake-Sleep Bayesian Program Learning*, 2020, [arXiv:2006.08381](#).
- [56] H. Wang, H. Xin, et al., *LEGO-Prover: Neural Theorem Proving with Growing Libraries*, ICLR 2024, [arXiv:2310.00656](#).
- [57] Y. Zhang, J. Sun, et al., *DreamProver: Evolving Transferable Lemma Libraries via a Wake-Sleep Theorem-Proving Agent*, 2026, [arXiv:2604.26311](#).
- [58] W. Ye, Z. Guan, et al., *ProofEvolve: Neuro-Symbolic Evolution for Formal Automated Theorem Proving*, 2026, [arXiv:2608.26334](#).

- [59] S. Böhme and T. Nipkow, *Sledgehammer: Judgement Day*, IJCAR 2010, LNCS 6173, pp. 107–121, [DOI](#).
- [60] I. Berlot-Attwell, T. Sesterhenn, F. Rudzicz, and X. Si, *Is This LLM Library Learning? Evaluation Must Account For Compute and Behaviour*, EACL 2026, pp. 3534–3568, [DOI](#).
- [61] Y. Fu, P. Liu, Z. Wang, and K. Yuan, *Proof-Refactor: Refactoring Generated Formal Proofs into Modular Artifacts*, 2026, [arXiv:2606.03743](#).
- [62] N. Onda et al., *LeanConjecturer: Automatic Generation of Mathematical Conjectures for Theorem Proving*, 2025, [arXiv:2506.22005](#).
- [63] D. Litt, *A beginning for mathematics*, 13 September 2026, [blog post](#).
- [64] T. Chu, *The AI dissenter viewpoint*, *Proofs and Prompts*, 9 August 2026, [blog post](#).
- [65] B. Green and forty-one other Fellows and Foreign Members of the Royal Society, *Open letter to Sir Paul Nurse, President of the Royal Society*, *Proofs and Prompts*, 17 September 2026, [blog post](#).
- [66] W. T. Gowers, *Why I didn't sign the Fields medallists' letter*, 17 September 2026, [blog post](#).
- [67] D. Koukoulopoulos, *A CERN for AI-assisted science?*, guest post on *What's new*, 17 September 2026, [blog post](#).
- [68] B. Antieau, *Fast math/slow math*, 15 September 2026, [blog post](#).
- [69] D. E. Knuth, T. Larrabee and P. M. Roberts. *Mathematical Writing*. Stanford report CS1193, based on the autumn 1987 course, Section 1.
- [70] Dayu Yang et al., *DocAgent: A Multi-Agent System for Automated Code Documentation Generation*, [arXiv:2504.08725v3](#), 2025, Sections 2–3.
- [71] Xincheng Wang, Ruida Hu, Cuiyun Gao, Pengfei Gao, and Chao Peng. *Evaluating Repository-level Software Documentation via Question Answering and Feature-Driven Development*. [arXiv:2604.06793v1](#), 2026.
- [72] Huajian Xin, Luming Li, Xiaoran Jin, Jacques Fleuriot, and Wenda Li. *APE-Bench: Evaluating Automated Proof Engineering for Formal Math Libraries*. [arXiv:2504.19110v3](#), 2026.
- [73] Auguste Poiroux, Gail Weiss, Viktor Kunčák, and Antoine Bosselut. *Reliable Evaluation and Benchmarks for Statement Autoformalization*. [arXiv:2406.07222v3](#), 2025.
- [74] T. Zhu, P. Monticone, J. Avigad and S. Welleck. *LeanArchitect: Automating Blueprint Generation for Humans and AI*. [arXiv:2601.22554v1](#), 2026, Methods and Case Studies.
- [75] M. Bolan et al. *The Equational Theories Project: Advancing Collaborative Mathematical Research at Scale*. [arXiv:2512.07087v2](#), 2025, Project Management and Data Management.
- [76] The mathlib Community. *The Lean Mathematical Library*. CPP 2020. DOI: [10.1145/3372885.3373824](#).
- [77] Google DeepMind. *Formal Conjectures*. [Repository README](#), consulted 29 September 2026; descriptive context only.
- [78] A. Novikov et al. *AlphaEvolve: A coding agent for scientific and algorithmic discovery*. [arXiv:2506.13131v1](#), 2025, Task Specification and Evolution.

- [79] B. Georgiev, J. Gómez-Serrano, T. Tao and A. Z. Wagner. *Mathematical exploration and discovery at scale*. [arXiv:2511.02864v3](#), 2025.
- [80] C. E. Brown, C. Kaliszyk and J. Urban. *Agent Hunt: Bounty Based Collaborative Autoformalization With LLM Agents*. [arXiv:2603.06737v1](#), 2026.
- [81] M. Kripner and M. Straka. *OpenProver: Agentic and Interactive Theorem Proving with Lean 4*. [arXiv:2607.09217v1](#), 2026, State and Memory Management.
- [82] D. Ma et al. *OProver: A Unified Framework for Agentic Formal Theorem Proving*. [arXiv:2605.17283v1](#), 2026, Introduction.
- [83] Z. Yang et al. *CircuitProver: Agentic Lean 4 Theorem Proving with Reusable Circuit Proof Library for Hardware Verification*. [arXiv:2607.27259v1](#), 2026.
- [84] H. Wijk et al. *RE-Bench: Evaluating frontier AI R&D capabilities of language model agents against human experts*. [arXiv:2411.15114v1](#), 2024; official abstract used here.
- [85] P. R. Halmos. *How to Write Mathematics*. L'Enseignement Mathématique 16 (1970), 123–152; sections on organising a mathematical presentation.
- [86] Various authors. *Writings on AI and Mathematics*. Supplied anthology, AI_ESSAYS_SOURCE.pdf, snapshot 22 September 2026; Kra, pp. 31–33; Riehl, pp. 39–40; Cohn, pp. 41–44; Gowers, pp. 55–60; Sanderson, pp. 73–77. Riehl’s essay is also available as [Why I do mathematical research](#).
- [87] L. Becker et al., *A Blueprint for the Formalization of Carleson’s Theorem on Convergence of Fourier Series*, 2025, [arXiv:2405.06423](#).
- [88] L. Aniva, C. Sun, B. Miranda, C. Barrett, and S. Koyejo, *Pantograph: A Machine-to-Machine Interaction Interface for Advanced Theorem Proving, High Level Reasoning, and Data Extraction in Lean 4*, in *Tools and Algorithms for the Construction and Analysis of Systems*, 2025, pp. 116–137, [DOI](#).
- [89] S. Kurgan et al., *TheoremGraph: Bridging Formal and Informal Mathematics*, 2026, [arXiv:2606.25363](#).
- [90] J. Asher, *LeanExplore: A Search Engine for Lean 4 Declarations*, 2025, [arXiv:2506.11085](#).
- [91] G. Gao et al., *LeanSearch v2: Global Premise Retrieval for Lean 4 Theorem Proving*, 2026, [arXiv:2605.13137](#).
- [92] A. Baanen, M. R. Ballard, J. Commelin, B. Gin-ge Chen, M. Rothgang, and D. Testa, *Growing Mathlib: Maintenance of a Large Scale Mathematical Library*, in *Intelligent Computer Mathematics*, 2025, [arXiv:2508.21593](#).
- [93] B. Yanahama and A. Sannai, *Lean Atlas: An Integrated Proof Environment for Scalable Human–AI Collaborative Formalization*, 2026, [arXiv:2604.16347](#).
- [94] R. Souza, T. Poteet, B. Etz, D. Rosendo, A. Gueroudji, W. Shin, P. Balaprakash, and R. Ferreira da Silva, *LLM Agents for Interactive Workflow Provenance: Reference Architecture and Evaluation Methodology*, WORKS at the ACM/IEEE International Conference for High Performance Computing, Networking, Storage and Analysis, 2025, [DOI](#).
- [95] S. Abedu, S. Khatoonabadi, and E. Shihab, *Synergizing LLMs and Knowledge Graphs: A Novel Approach to Software Repository-Related Question Answering*, 2024, revised 2025, [arXiv:2412.03815](#).

- [96] T. Tao, *Mathematics in the age of AI*, 2026, [arXiv:2608.16753](#).
- [97] D. P. Anderson, *BOINC: A Platform for Volunteer Computing*, *Journal of Grid Computing* 18 (2020), 99–122, [DOI](#).
- [98] Great Internet Mersenne Prime Search, *GIMPS*, project documentation and discovery-credit record, [mersenne.org](#), accessed August 2026.
- [99] Polymath Project, *General polymath rules*, [project rules](#), accessed August 2026.
- [100] Lean community, *Contributing to mathlib*, [contributor guide](#), accessed August 2026.
- [101] Palomar Registry, *About Palomar and Contribution policy*, [registry documentation](#) and [submission standard](#), accessed August 2026.
- [102] J. Henkel, *The Mathematician’s Assistant: Integrating AI into Research Practice*, 2025, [arXiv:2508.20236](#).
- [103] J. Dekoninck et al., *The Open Proof Corpus: A Large-Scale Study of LLM-Generated Mathematical Proofs*, 2025, [arXiv:2506.21621](#).
- [104] NISO, *CRedit: Contributor Roles Taxonomy*, [role definitions](#), accessed August 2026.
- [105] GitHub, *Preventing pwn requests*, GitHub Actions security guidance, [documentation](#), accessed August 2026.
- [106] M. Yuan et al., *End-to-End Testing of Open-Source Hardware Documentation Developed in Large Collaborations*, 2023, [arXiv:2309.05942](#).